



**University of
Sheffield**

COM31006 Computer Vision Assignment Report

Vivek Vijay Choradia

May 25, 2024

1 Introduction

This report is structured to provide a comprehensive account of each stage of the COM31006 Computer Vision assignment. It begins with the construction of an image dataset, followed by the implementation of feature detection and description methods, feature matching techniques, and concludes with image stitching. Each section is dedicated to a specific component of the assignment.

- **Image Dataset Construction:** This section describes the process of collecting and preparing a dataset consisting of pairs of images that have a substantial overlap of content. This sections gives an overview of the methodology used for ensuring the quality and relevance of the dataset.
- **Feature Detection:**Two feature selection methods have been implemented and compared : the Harris corner detection method and the Scale-Invariant Feature Transform (SIFT) method. The section provides a detailed descriptions of the algorithms and their implementations.
- **Feature Description:** This section covers the implementation of feature descriptors for the detected features. Two descriptors were implemented: SIFT descriptor and a low-cost binary descriptor.
- **Feature Matching:** The matching of features between images is addressed using methods such as the Sum of Squared Differences (SSD) and the ratio test. The implementation details and performance evaluation of these methods are discussed.
- **Image Stitching:** This section discussed the process of stitching images together. This is done by finding the homography that relates two images and warping them to create a seamless panoramic image. The implementation and results of the image stitching process are also presented.

In addition to the technical implementation, a user interface (UI) is developed to visualize the results of feature detection, feature matching, and image stitching. This UI serves as a tool for both debugging and demonstrating the functionality of the implemented algorithms.

The subsequent sections will provide detailed explanations, experimental results, and visual illustrations for each stage of the assignment.

2 Image Dataset Construction

Constructing an image dataset is a critical foundation for the successful implementation of feature detection, description, and matching algorithms. This section outlines the methodology used for collecting and preparing the image dataset utilized in this assignment.

To create a comprehensive and diverse dataset, ten pairs of images were collected, including one provided as part of the assignment brief. Each pair of images was chosen to have at least 70% overlapping content. This was done to ensure sufficient common features for the subsequent tasks of feature detection and matching. The images were acquired using an iPhone 12's camera. Efforts were made to ensure that image pairs were captured under uniform lighting conditions and from consistent viewpoints.

The final dataset consists of five pairs of images (and 5 other images for testing), each pair exhibiting significant overlap in content. The images cover a range of scenes including urban landscapes, natural environments, and indoor settings. This variety ensures that feature detection and matching algorithms are evaluated under various real-world conditions.

3 Feature Detection

Feature detection enables the identification of distinctive and invariant points within an image. This section describes the implementation



Figure 1: Sample pairs of images from the constructed dataset.

of two feature detection methods: Harris corner detection and Scale-Invariant Feature Transform (SIFT). Both methods are evaluated for their effectiveness in identifying salient features.

3.1 Harris Corner Detection

The Harris corner detection algorithm is based on the concept of detecting corners by analyzing the changes in intensity in all directions. Corners are identified as points where the image gradients have large variations. The mathematical foundation of this method involves the computation of the Harris matrix, defined as:

$$M = \sum_{u,v} w(u,v) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \quad (1)$$

where I_x and I_y are the image gradients in the x and y directions, and $w(u,v)$ is a window function, typically a Gaussian.

The corner response function R is calculated as:

$$R = \det(M) - k(\text{trace}(M))^2 \quad (2)$$

where k is a sensitivity parameter, typically set to 0.04 to 0.06. Corners are identified where R is above a certain threshold.

The Harris corner detector was implemented using the OpenCV library within the Image Processor class. The implementation leverages

OpenCV's built-in functions, which abstract some of the lower-level details involved in the Harris corner detection process. The key steps involved are:

- **Converting the Image to Grayscale and Floating Point:** Since the Harris corner detection requires single-channel float input, the original image is first converted to grayscale and then to a floating point format.
- **Applying the Harris Corner Detection Function:** The `cv2.cornerHarris()` function is utilized, which internally computes the image gradients, the Harris matrix M , and the corner response function R . This function requires specifying parameters such as the neighborhood size, aperture size for the Sobel operator, and the Harris detector free parameter (k).
- **Dilating the Resultant Corner Response:** To emphasize the corners, the corner response image is dilated using `cv2.dilate()`, making the corner points more visible.
- **Thresholding for Strong Corners:** A threshold is applied to the dilated corner response to identify significant corners. Points with a response greater than 1% of the max-

imum response are considered strong corners.

- **Marking Detected Corners on the Image:** The identified strong corners are marked directly on the original image for visualization, typically colored in red for clear visibility.

This procedure uses OpenCV's optimized functions to efficiently detect corner. This reduced the need for explicit calculation of gradients and manual implementation of non-maximum suppression, which are handled internally by the `cv2.cornerHarris()` function.

3.2 Scale-Invariant Feature Transform (SIFT)

The SIFT algorithm is implemented using OpenCV. It is designed to detect and describe local features that are invariant to scale and rotation. The process involves several key mathematical steps:

- **Scale-Space Extrema Detection:** Key-points are identified as maxima and minima of the Difference of Gaussian (DoG) function applied in scale-space. This is expressed as:

$$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y)$$

where G is the Gaussian Blur function, I is the input image, σ is the scale, and k is a constant multiplicative factor.

- **Keypoint Localization:** Each candidate keypoint is refined to determine its precise location and scale. This involves a Taylor expansion of the scale-space function $D(x, y, \sigma)$ to fit a 3D quadratic function:

$$D(\mathbf{x}) \approx D + \frac{\partial D}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

where $\mathbf{x} = (x, y, \sigma)^T$ represents the offset relative to the detected feature location in scale-space.

- **Orientation Assignment:** Each keypoint is assigned one or more orientations based on local image gradient directions. The gradient magnitude $m(x, y)$ and orientation $\theta(x, y)$ are used to form a histogram from the gradients within a region around the keypoint:

$$m(x, y) = \sqrt{\left(\frac{\partial L}{\partial x}\right)^2 + \left(\frac{\partial L}{\partial y}\right)^2}$$

$$\theta(x, y) = \tan^{-1}\left(\frac{\partial L}{\partial y} / \frac{\partial L}{\partial x}\right)$$

- **Keypoint Descriptor:** The local image descriptors are formed from the gradients around each keypoint. The gradients are weighted by a Gaussian window and accumulated into orientation histograms summarizing the contents over subregions.

Implementation Using OpenCV:

- **Initialization:** A SIFT object is created with parameters specifying the number of features, contrast threshold, and edge threshold using the command:

```
sift = cv2.SIFT_create(nfeatures=500,
contrastThreshold=0.08,
edgeThreshold=10)
```

- **Detection and Description:** Keypoints and descriptors are computed from the grayscale image using:

```
keypoints, descriptors =
sift.detectAndCompute(self.gray_image,
None)
```

- **Visualization:** Keypoints are visualized on the original image to enhance visibility and facilitate analysis:

```
self.image =
cv2.drawKeypoints(self.image,
self.keypoints, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```



Figure 2: Harris corners detected on image pair 1.



Figure 3: SIFT keypoints detected in image pair 1.

3.3 Comparison and Analysis

To evaluate the performance of the Harris and SIFT methods, both algorithms were applied to the image dataset to analyze the result. This comparative analysis is provided in Table 1.

Despite Harris corner detection showing a higher number of features detected and lower computational time, it is essential to consider the nature of these features. Harris algorithm detects corners based on intensity changes and does not account for scale or rotation, making these features less robust compared to those detected by SIFT. SIFT, although computationally more intensive, detects features that are invariant to scale and rotation. This

makes SIFT features more adaptable to diverse application scenarios where images may undergo significant transformations.

In conclusion, while Harris may be preferred for scenarios demanding speed over feature robustness, such as simple video tracking or real-time motion detection, SIFT's robust features are more suited for complex image processing tasks like object recognition, image matching across varied scales and orientations, and detailed panoramic image stitching. Thus, SIFT becomes the ideal choice of algorithm for subsequent tasks.

Table 1: Comparison of Harris and SIFT Feature Detection Methods in image pair 1.

Criterion	Harris Corner	SIFT
Number of Features	(16278, 13987)	(1001, 1000)
Computational Time (for image pair)	(0.01s, 0.01s)	(0.05s, 0.06s)

4 Feature Description

Feature description involves generating unique representations for the detected features. These representations, or descriptors, allow comparison and matching of features across different images. This section describes the methods used for feature description, detail their implementation, and present the results with illustrations for both ORB and SIFT descriptors.

4.1 Methods

The goal of feature description is to create a compact and distinctive representation of each detected feature. For this assignment, Two types of descriptors implemented : the SIFT descriptor and the ORB (Oriented FAST and Rotated BRIEF) descriptor.

4.1.1 SIFT Descriptor

The Scale-Invariant Feature Transform (SIFT) descriptor is designed to be invariant to scale, rotation, and illumination changes. It achieves this by capturing the gradient information around each keypoint and organizing it into a histogram of gradient orientations.

- **Gradient Computation:** For each keypoint, the image gradients are computed in the surrounding region.
- **Orientation Assignment:** The gradients are rotated to align with the keypoint's dominant orientation.
- **Histogram Generation:** The rotated gradients are divided into a grid of cells, and a histogram of gradient orientations is computed for each cell.

- **Descriptor Formation:** The histograms from all cells are concatenated to form the final descriptor vector.

4.1.2 ORB Descriptor

ORB is an efficient alternative to SIFT that combines the FAST keypoint detector and the BRIEF descriptor, with modifications to enhance performance and provide orientation invariance.

- **FAST Keypoint Detection:** ORB starts by detecting keypoints using the FAST algorithm.
- **Orientation Assignment:** Each keypoint is assigned an orientation based on the intensity centroid, making the descriptor rotation invariant.
- **BRIEF Descriptor Calculation:** The BRIEF descriptor is computed around each keypoint, but with orientation compensation to account for the assigned orientation.
- **rBRIEF (Rotated BRIEF):** To improve robustness, ORB uses a modified version of BRIEF that rotates the sampling pattern according to the keypoint orientation.

ORB stands out among other low-cost descriptors for several reasons. ORB is designed to be fast. The use of the FAST detector and the BRIEF descriptor ensures that keypoints and descriptors can be computed quickly, making ORB suitable for real-time applications. The orientation assignment step in ORB makes it rotation invariant, which is a significant advantage over traditional BRIEF that lacks this property. The rBRIEF descriptor enhances the robustness of the feature matching

process, improving performance in various conditions such as changes in viewpoint and illumination. These characteristics make ORB a compelling choice for many computer vision applications where speed and efficiency are critical.

The implementation of both the SIFT and ORB descriptors was carried out using OpenCV. Same implementation of SIFT from feature detection was used as it provided both the parameters - keypoints and descriptors. The ORB descriptor does the following steps to provide descriptors:

- **Keypoint Detection:** Detect keypoints using the FAST algorithm.
- **Orientation Assignment:** Compute the orientation for each keypoint using the intensity centroid method.
- **Descriptor Computation:** Calculate the BRIEF descriptor for each keypoint, compensating for the keypoint’s orientation.

4.2 Results

The feature descriptions generated by both methods were evaluated using the constructed dataset. SIFT descriptors provide a rich and distinctive representation of features, while ORB descriptors offer a balance between computational efficiency and descriptor distinctiveness.

Table 2 compares the performance metrics of SIFT and ORB descriptors in terms of descriptor size and computational time.

Table 2: Comparison of SIFT and ORB Feature Descriptors

Criterion	SIFT	ORB
Descriptor Size	(1000,128)	(500, 32)
Computation Time	0.238s	0.013s

On comparing both the descriptors, the following conclusion was made: Despite the higher computational cost of SIFT compared to ORB, its unmatched accuracy in feature description across scale and rotation variations makes it indispensable for the project’s needs. The ability of SIFT to provide reliable and highly distinctive

descriptors underpins its selection for subsequent tasks, particularly where precision is critical.

5 Feature Matching

Feature matching involves finding correspondences between features in different images. This process is essential for applications such as image stitching, where accurate matching of features is required to align and merge images. This section explains the methods used for feature matching.

5.1 Methods

The objective of feature matching is to identify pairs of corresponding features between two images. For this two methods of feature matching were implemented: the Sum of Squared Differences (SSD) and the ratio test.

5.1.1 Sum of Squared Differences (SSD)

The SSD method calculates the similarity between two feature descriptors by computing the sum of squared differences between their corresponding elements. The feature with the smallest SSD value is considered the best match.

$$d(X, Y) = \sum_{i=1}^n (X_i - Y_i)^2 \quad (3)$$

where X and Y are the feature descriptors, and n is the length of the descriptors.

5.1.2 Ratio Test

The ratio test is used to improve the robustness of the matching process. For each feature in the first image, the SSD is calculated for the two closest matches in the second image. The ratio of the smallest SSD to the second smallest SSD is computed, and matches are accepted if this ratio is below a certain threshold (in our case, it is set to 0.75).

$$\text{Ratio} = \frac{\text{SSD}_{\text{nearest}}}{\text{SSD}_{\text{second nearest}}} \quad (4)$$

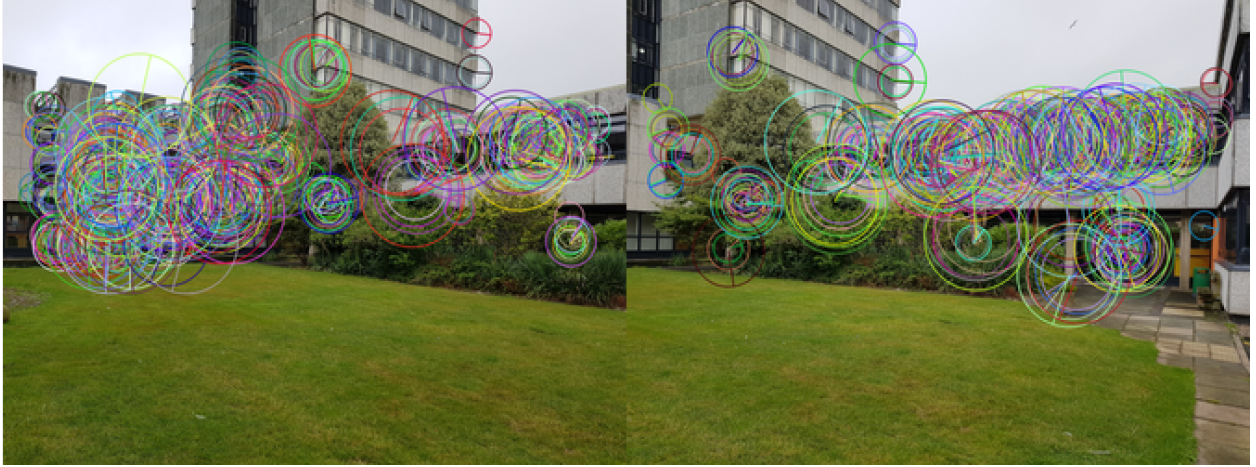


Figure 4: ORB keypoints and descriptors for detected features for image pair 1.

The ratio test helps eliminate ambiguous matches and improves the accuracy of the feature matching process.

5.2 Implementation

The steps involved in implementing feature matching are as follows:

- **Descriptor Extraction:** Extract feature descriptors for both images using the SIFT and/or ORB methods.
- **Matching Descriptors:** Used the ‘cv::BFMatcher’ class in OpenCV to match descriptors between the two images.
- **Applying Ratio Test:** Used ‘cv::BFMatcher::knnMatch’ for finding the k-nearest neighbors (k=2) required for the ratio test. Then manually filtered out weak matches.

5.3 Results

The evaluation of feature matching was conducted using a carefully constructed dataset, with the outcomes visualized through graphical representations of line connections between corresponding features across image pairs. These visual aids facilitate a clearer understanding of the matching efficacy and the spatial relationships between matched features.

The quantitative analysis of the feature matching performance, utilizing SIFT and ORB descriptors, is summarized in Table 3. This table provides key metrics, including the number of successful matches and the accuracy of these matches.

Table 3: Comparative Analysis of Feature Matching Performance using SIFT and ORB Descriptors on image pair 1.

Criterion	SIFT	ORB
Number of Matches	159	36
Matching Accuracy	32%	7%

The comparative evaluation reveals a significant discrepancy in matching accuracy between the two methods. The SIFT algorithm demonstrates a substantially higher number of correct matches and a greater matching accuracy of 32%, compared to only 7% for ORB. Despite ORB’s faster computational time, its lower matching accuracy underscores its limitations in capturing essential features necessary for precise matching. Consequently, based on the comprehensive performance metrics and considering the critical requirement for reliability in feature matching, the SIFT descriptor is considered more suitable for subsequent tasks in this project.



Figure 5: Feature matching using SIFT descriptors between pairs of images (Pair 1).



Figure 6: Feature matching using ORB descriptors between pairs of images (Pair 1).

6 Image Stitching

Image stitching refers to the process of combining multiple photographic images with overlapping fields of view to produce a segmented panorama or high-resolution image. This section details the methodology for combining two images that maintain at least a 70% overlap, discusses the implemented techniques for image stitching, and presents the results with accompanying illustrations.

6.1 Methods

The image stitching process involves several critical steps: identifying correspondences between

images, estimating the transformation (homography) to align the images, and blending the images to achieve a seamless panorama.

6.1.1 Feature Matching

The initial step in image stitching is feature matching using SIFT (Scale-Invariant Feature Transform), which is crucial for accurate homography estimation between the images.

6.1.2 Homography Estimation

Upon identifying matching features, the next step involves computing the homography matrix H , a 3×3 transformation matrix. This matrix is

estimated using the Direct Linear Transformation (DLT) algorithm and refined with RANSAC (Random sample consensus) to remove outliers and ensure robust estimation.

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \quad (5)$$

6.1.3 Image Warping and Blending

Following the estimation of the homography matrix, the second image is warped to align with the first. Subsequently, the images are blended to ensure a seamless transition, addressing potential issues such as exposure differences and blending artifacts.

6.2 Implementation

6.2.1 Homography Estimation Implementation

The procedures for estimating the homography include:

- Extracting the coordinates of matched key-points from the images.
- Utilizing the ‘cv2.findHomography’ function with RANSAC (Random sample consensus) to compute the homography matrix.

6.2.2 Image Warping and Blending Implementation

The procedures for warping and blending the images encompass:

- Applying the homography to warp the second image using the ‘cv2.warpPerspective’ function.
- Combining the images on an output canvas adequately sized to accommodate both images.

6.3 Results

The results of the image stitching were assessed visually on a dataset designed to evaluate

the accuracy of stitching and seam visibility. All the image pairs were stitched and have been saved in *output* directory. Figure 7 provides an example of stitched image outputs.

The visual inspection confirmed successful stitching with minimal visible seams and accurate alignment. For some image pairs, where the images did not have exactly same orientation, even those images were handled well. Table 4 provides a summary of performance metrics, including computation time and quality of the stitching.

Table 4: Performance Metrics of Image Stitching

Criterion	Value
Average Computation Time (per image pair)	0.37 s
Stitching Quality (subjective score)	High

7 Conclusion

This report detailed the implementation and evaluation of different methods for feature detection, feature description, feature matching, and image stitching using OpenCV and Python. The key points and findings are summarized below:

- **Image Dataset Construction:** A diverse dataset of image pairs with significant overlapping content was collected and pre-processed. This dataset provided a robust foundation for evaluating the feature detection and matching algorithms.
- **Feature Detection:** The Harris corner detection and SIFT (Scale-invariant feature transform) feature detection methods were implemented and evaluated. SIFT was found to be more robust and invariant to scale and rotation, though it was computationally more intensive than the Harris method.
- **Feature Description:** Both SIFT and ORB (Oriented FAST and Rotated BRIEF) descriptors were implemented. SIFT provided high distinctiveness and invariance



Figure 7: Stitched image result from the dataset - image pair 1.

but at a higher computational cost. ORB offered a balanced trade-off between efficiency and robustness.

- **Feature Matching:** Feature matching was performed using Sum of Squared Differences (SSD) and the ratio test. The ratio test improved matching accuracy by eliminating ambiguous matches. Both SIFT and ORB descriptors were effective, with SIFT achieving higher accuracy.
- **Image Stitching:** The homography matrix was estimated using RANSAC (Random sample consensus), and image warping and blending were performed to create stitch images. The methods demonstrated effective alignment and seamless blending, and hence resulting in high-quality stitched images.

Overall, the implemented methods provided solutions for feature detection, description, matching, and image stitching, achieving the objectives of the assignment.